LAPORAN JOBSHEET 7

PRAKTIKUM PEMROGRAMAN BERBASIS OBJEK

Disusun oleh:

Nama : Elsa Marthalinda

NIM : 254107020204

Kelas : 2B

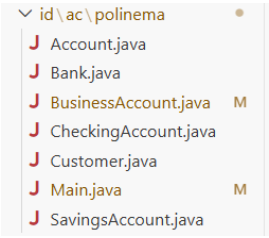
PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

OKTOBER 2026

A. Struktur Project



Gambar 1: Struktur folder ide.ac.polinema

Folder proyek ini berisi tujuh berkas kode sumber Java yang saling terhubung untuk mensimulasikan sistem perbankan berbasis Pemrograman Berorientasi Objek (OOP). Di dalamnya terdapat Account.java sebagai kelas induk (superclass) pengelola data rekening dasar, yang diturunkan ke dalam tiga jenis rekening spesifik: SavingsAccount.java (rekening tabungan), CheckingAccount.java (rekening cek), dan BusinessAccount.java (rekening bisnis). Selain itu, terdapat Customer.java untuk mengelola informasi data nasabah dan Bank.java sebagai pusat pengelola seluruh data nasabah beserta rekeningnya. Seluruh sistem ini dikendalikan oleh Main.java sebagai kelas penggerak (driver class) yang mengeksekusi dan menguji alur interaksi transaksi perbankan secara keseluruhan.

B. Implementasi & Menjalankan Program

Langkah 1: Account Menyediakan Titik untuk Di-override

```java
package id.ac.polinema;
public class Account {
    private String accountNumber;
    private Customer owner;
    private double balance;

    public Account(String accountNumber, Customer owner, double balance) {
        this.accountNumber = accountNumber;
        this.owner = owner;
        this.balance = balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public Customer getOwner() {
        return owner;
    }

    public double getBalance() {
        return balance;
    }

    public boolean deposit(double amount) { // jika uang jika kurang dari sama dengan 0 maka return false
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }

    public boolean withdraw(double amount) {
        if (!canWithdraw(amount)) {
            return false;
        }
        balance -= amount;
        return true;
    }

    protected boolean canWithdraw(double amount) {
        return amount > 0 && amount <= balance;
    }

    public void printInfo() {
        System.out.println(accountNumber + " - " + owner.getName() + " - balance: " + balance);
    }
}
```

Gambar 2: Account.java, withdraw() memanggil canWithdraw() yang baru

Kelas Account bertindak sebagai kelas induk (*superclass*). Kelas ini memiliki atribut privat accountNumber, owner, dan balance. Di dalamnya terdapat metode deposit(), withdraw(), serta metode proteksi canWithdraw(). Metode withdraw() menggunakan pola desain *Template Method* kecil, di mana validasi penarikan didelegasikan ke metode canWithdraw().

Konsep OOP/Java yang diterapkan:

- **Enkapsulasi** Semua atribut diset menjadi private dan diakses melalui metode *getter* (getAccountNumber(), getOwner(), getBalance()) serta pembaruan saldo melalui metode deposit() dan withdraw()
- **Access Modifier protected:** Metode canWithdraw(double amount) dideklarasikan dengan hak akses protected

```
A001 - Nadia - balance: 350000.0
A002 - Sari - balance: 200000.0
```

Gambar 3: Hasil output pemanggilan canWithdraw()

Output menampilkan informasi akun dasar berupa nomor rekening, nama pemilik, dan sisa saldo untuk rekening Nadia (A001) dan Sari (A002). **Object State & Representation:** Menampilkan representasi status internal (*state*) dari objek Account setelah metode printInfo() dipanggil.

Langkah 2: Bank Mengelola Banyak Account

```java
package id.ac.polinema;
public class SavingsAccount extends Account {
    private static final double MINIMUM_BALANCE = 50000;
    private double interestRate;

    public SavingsAccount(String accountNumber, Customer owner, double balance, double interestRate) {
        super(accountNumber, owner, balance);
        this.interestRate = interestRate;
    }

    public double getInterestRate() {
        return interestRate;
    }

    @Override
    protected boolean canWithdraw(double amount) {
        return amount > 0 && (getBalance() - amount) >= MINIMUM_BALANCE;
    }

    @Override
    public void printInfo() {
        super.printInfo();
        System.out.println("Account type: Savings, interest rate: " + interestRate);
    }
}
```

Gambar 4: SavingsAccount.java meng-override canWithdraw dan printInfo

Kelas SavingsAccount mewarisi kelas Account (extends Account). Kelas ini menetapkan batasan saldo minimum (MINIMUM_BALANCE = 50000) dan memiliki

atribut interestRate. Metode canWithdraw() di-override untuk memastikan saldo setelah penarikan tidak turun di bawah Rp50.000,00. Metode printInfo() di-override untuk menambahkan informasi jenis rekening dan suku bunga.

Konsep OOP/Java yang diterapkan:

- **Inheritance (Pewarisan):** Penggunaan kata kunci extends menjadikan SavingsAccount sebagai *subclass* yang mewarisi sifat dan metode dari Account.
- **Constructor Chaining (super keyword):** Pemanggilan super(accountNumber, owner, balance) pada konstruktor memastikan inisialisasi atribut kelas induk diproses terlebih dahulu sebelum menginisialisasi atribut spesifik milik SavingsAccount.
- **Method Overriding (@Override annotation):** Menyediakan implementasi baru untuk metode canWithdraw() dan printInfo() yang sudah ada pada kelas induk.

Perbarui main.java:

```java
package id.ac.polinema;
public class Main {
    Run | Debug
    public static void main(String[] args) {
        Customer customer = new Customer(name: "Rian", phone: "0812-0000-0003");
        SavingsAccount savings = new SavingsAccount(accountNumber: "A003", customer, balance: 100000, interestRate: 0.02);

        boolean result = savings.withdraw(amount: 70000);
        System.out.println("Withdraw 70000 allowed? " + result);
        savings.printInfo();
    }
}
```

Gambar 5: Main.java menguji penarikan yang melanggar saldo minimum

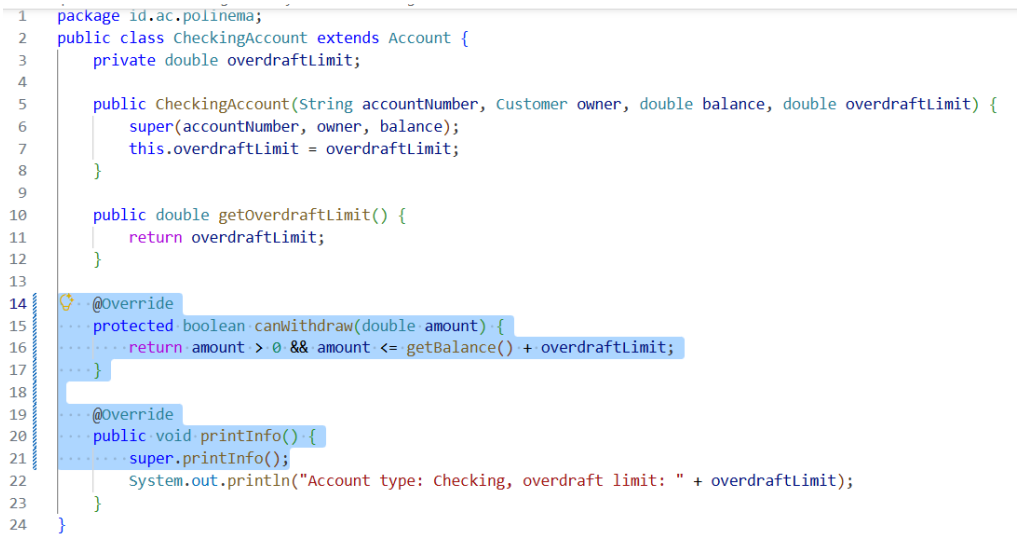
Kode membuat objek Customer bernama "Rian" dan objek SavingsAccount dengan saldo awal Rp100.000,00 dan bunga 0.02. Kemudian dicoba penarikan dana sebesar Rp70.000,00.

```
Withdraw 70000 allowed? false
A003 - Rian - balance: 100000.0
Account type: Savings, interest rate: 0.02
```

Gambar 6: Hasil output menggunakan Override canWithdraw

Program mencetak Withdraw 70000 allowed? false. Saldo akun tetap Rp100.000,00 dan informasi detail akun menampilkan suku bunga 0.02. Menunjukkan keberhasilan pengisian logika bisnis pada *subclass* yang mencegah pengurangan saldo tidak sah melalui mekanisme *method overriding*.

Langkah 3: CheckingAccount Meng-override canWithdraw dan printInfo



```java
package id.ac.polinema;
public class CheckingAccount extends Account {
    private double overdraftLimit;

    public CheckingAccount(String accountNumber, Customer owner, double balance, double overdraftLimit) {
        super(accountNumber, owner, balance);
        this.overdraftLimit = overdraftLimit;
    }

    public double getOverdraftLimit() {
        return overdraftLimit;
    }

    @Override
    protected boolean canWithdraw(double amount) {
        return amount > 0 && amount <= getBalance() + overdraftLimit;
    }

    @Override
    public void printInfo() {
        super.printInfo();
        System.out.println("Account type: Checking, overdraft limit: " + overdraftLimit);
    }
}
```

Gambar 7: CheckingAccount.java meng-override canWithdraw dan printInfo

Kelas CheckingAccount mewarisi Account dan menambahkan atribut overdraftLimit. Metode canWithdraw() di-override sehingga penarikan diizinkan selama jumlah penarikan tidak melebihi akumulasi saldo saat ini ditambah batas *overdraft* (getBalance() + overdraftLimit).

Konsep OOP/Java yang diterapkan:

- **Method Overriding:** Mengubah aturan bisnis validasi transaksi sesuai dengan karakteristik rekening giro (*checking account*) yang mengizinkan saldo negatif hingga batas tertentu.
- **Super Method Call (super.printInfo()):** Memanggil fungsi cetak dari kelas induk terlebih dahulu sebelum menambahkan output spesifik overdraftLimit.

Perbarui main.java;



```java
package id.ac.polinema;
public class Main {
    Run | Debug
    public static void main(String[] args) {
        Customer customer1 = new Customer(name: "Rian", phone: "0812-0000-0003");
        SavingsAccount savings = new SavingsAccount(accountNumber: "A003", customer1, balance: 100000, interestRate: 0.02);
        savings.withdraw(amount: 70000);

        Customer customer2 = new Customer(name: "Dewi", phone: "0812-0000-0004");
        CheckingAccount checking = new CheckingAccount(accountNumber: "A004", customer2, balance: 100000, overdraftLimit: 200000);
        boolean result = checking.withdraw(amount: 250000);
        System.out.println("Withdraw 250000 allowed? " + result);

        Bank bank = new Bank(capacity: 10);
        bank. addAccount (savings) ;
        bank.addAccount (checking);
        bank.printAllAccounts();
    }
}
```

Gambar 8: Main.java menguji penarikan melebihi saldo lewat overdraft

Membuat objek CheckingAccount untuk pelanggan "Dewi" dengan saldo Rp100.000,00 dan overdraftLimit Rp200.000,00. Dilakukan penarikan Rp250.000,00. Objek savings dan checking kemudian dimasukkan ke dalam koleksi objek Bank. Objek

dari tipe SavingsAccount dan CheckingAccount diperlakukan secara seragam ketika ditambahkan ke dalam objek Bank yang menyimpan tipe referensi dasar Account.

```
Withdraw 250000 allowed? true
A003 - Rian - balance: 100000.0
Account type: Savings, interest rate: 0.02
A004 - Dewi - balance: -150000.0
Account type: Checking, overdraft limit: 200000.0
```

Gambar 9: Hasil output hasil Main.java

Penarikan Rp250.000,00 pada CheckingAccount diizinkan (true) karena total batas dana tersedia adalah $100.000 + 200.000 = 300.000$. Saldo berkurang menjadi -150000.0 (kondisi *overdraft*).

Langkah 4: Overloading, deposit() dengan Catatan

```java
package id.ac.polinema;
public class Account {
    private String accountNumber;
    private Customer owner;
    private double balance;

    public Account(String accountNumber, Customer owner, double balance) {
        this.accountNumber = accountNumber;
        this.owner = owner;
        this.balance = balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public Customer getOwner() {
        return owner;
    }

    public double getBalance() {
        return balance;
    }

    public boolean deposit(double amount) { // jika uang jika kurang dari sama dengan 0 maka return false
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }

    public boolean withdraw(double amount) {
        if (!canWithdraw(amount)) {
            return false;
        }
        balance -= amount;
        return true;
    }

    protected boolean canWithdraw(double amount) {
        return amount > 0 && amount <= balance;
    }

    public boolean deposit(double amount, String note) {
        if (!deposit(amount)) {
            return false;
        }
        System.out.println(accountNumber + " deposit note: " + note);
        return true;
    }

    public void printInfo() {
        System.out.println(accountNumber + " - " + owner.getName() + " - balance: " + balance);
    }
}
```

Gambar 10: Account.java, deposit(double, String) sebagai overload dari deposit(double)

Di dalam kelas Account, ditambahkan metode baru public boolean deposit(double amount, String note). Metode ini menerima dua parameter dan memanggil metode deposit(amount) internal, lalu mencetak catatan deposit (*deposit note*).

Konsep OOP/Java yang diterapkan:

- **Method Overloading (Polimorfisme):** Dalam kelas yang sama terdapat dua metode dengan nama persis sama (deposit), namun memiliki jumlah atau jenis tipe parameter berbeda (*method signature* berbeda: deposit(double) vs deposit(double, String)).
- **Method Chaining:** Metode deposit dengan catatan memanfaatkan metode deposit dasar untuk memproses penambahan saldo sehingga menghindarkan duplikasi kode penambahan saldo.

Perbarui Main.java:

```
1  package id.ac.polinema;
2  public class Main {
       Run | Debug
3      public static void main(String[] args) {
4          Customer customer1 = new Customer(name: "Rian", phone: "0812-0000-0003");
5          SavingsAccount savings = new SavingsAccount(accountNumber: "A003", customer1, balance: 100000, interestRate: 0.02);
6
7          savings.deposit(amount: 50000, note: "Initial top-up");
8          savings.printInfo();
9      }
10 }
```

Gambar 11: Main.java memanggil deposit dengan catatan

Pada kode ini, dibuat objek SavingsAccount untuk pelanggan "Rian" dengan nomor rekening "A003", saldo awal sebesar Rp100.000,00, dan suku bunga 0.02. Selanjutnya, dilakukan pemanggilan metode savings.deposit(50000, "Initial top-up") yang melewatkan dua buah argumen, yaitu nilai nominal deposit berjenis double (50000) dan pesan catatan berjenis String ("Initial top-up").

Konsep OOP/Java yang diterapkan:

- **Compile-time Polymorphism:** Saat baris kode savings.deposit(50000, "Initial top-up") ditulis dan dikompilasi, kompiler Java secara otomatis mencocokkan jumlah dan tipe data parameter yang dilewatkan.
- **Method Delegation:** Di dalam implementasi deposit dua parameter tersebut, eksekusi pembaruan saldo divalidasi dan didelegasikan kembali ke metode deposit(amount) dasar.

```
A003 deposit note: Initial top-up
A003 - Rian - balance: 150000.0
Account type: Savings, interest rate: 0.02
```

Gambar 12: Output penerapan overloading di Account.java

Output mencetak catatan transaksi A003 deposit note: Initial top-up diikuti pembaruan saldo rekening A003 yang bertambah dari Rp100.000,00 menjadi Rp150.000,00 (balance: 150000.0) beserta detail tipe akun dan suku bunganya.

Konsep OOP/Java yang diterapkan:

- **Method Overload:** Membuktikan bahwa metode deposit(double, String) berhasil dipanggil untuk mencetak catatan transaksi sekaligus memperbarui saldo objek.
- Menunjukkan perubahan nilai saldo internal pada objek SavingsAccount tersimpan dengan benar saat informasi akun dicetak kembali.

Tugas dan Deliverable

1. Tugas mandiri:

```java
package id.ac.polinema;
public class BusinessAccount extends Account {
    private static final double MINIMUM_BALANCE = 1000000;
    private double monthlyTransactionFee;

    public BusinessAccount(String accountNumber, Customer owner, double balance, double monthlyTransactionFee) {
        super(accountNumber, owner, balance);
        this.monthlyTransactionFee = monthlyTransactionFee;
    }

    public double getMonthlyTransactionFee() {
        return monthlyTransactionFee;
    }

    @Override
        protected boolean canWithdraw(double amount) {
            return amount > 0 && (getBalance() - amount) >= MINIMUM_BALANCE;
        }

        @Override
        public void printInfo() {
            super.printInfo();
            System.out.println("Account type: Business, monthly fee: " + monthlyTransactionFee);
        }
}
```

Gambar 13: BusinessAccount.java meng-override canWithdraw dan printInfo

Kelas BusinessAccount dibuat menurunkan Account dengan aturan khusus: saldo minimum yang harus disisa adalah Rp1.000.000,00 (MINIMUM_BALANCE = 1000000) dan terdapat biaya transaksi bulanan (monthlyTransactionFee).

Konsep OOP/Java yang diterapkan:

- **Inheritance:** Menerapkan spesialisasi bisnis pada rekening bisnis dengan memperketat aturan saldo minimum melalui *method overriding* pada canWithdraw().

```java
package id.ac.polinema;
public class Main {
    Run | Debug
    public static void main(String[] args) {
        Customer customer = new Customer(name: "Rian", phone: "0812-0000-0003");
        BusinessAccount business = new BusinessAccount(accountNumber: "A003", customer, balance: 2000000, monthlyTransactionFee… 50000);

        boolean result = business.withdraw(amount: 1500000);
        System.out.println("Withdraw 1500000 allowed? " + result);

        business.printInfo();
    }
}
```

Gambar 14: Main.java menguji penarikan pada BusinessAccount

Kode ini membuat objek Customer bernama "Rian" dan objek BusinessAccount dengan nomor rekening "A003", saldo awal Rp2.000.000,00, serta biaya transaksi bulanan sebesar Rp50.000,00. Selanjutnya, program mencoba melakukan penarikan dana sebesar Rp1.500.000,00 melalui baris business.withdraw(1500000) dan mencetak status keberhasilan transaksi beserta detail akun menggunakan business.printInfo()

```
Withdraw 1500000 allowed? false
A003 - Rian - balance: 2000000.0
Account type: Business, monthly fee: 50000.0
```

Gambar 15: Hasil output uji coba BusinessAccount

Menampilkan status penarikan Withdraw 1500000 allowed? false yang menandakan transaksi ditolak. Baris berikutnya mencetak informasi akun di mana saldo rekening tetap utuh sebesar Rp2.000.000,00 (balance: 2000000.0) disertai rincian tipe akun dan biaya transaksi bulanan monthly fee: 50000.0.

Pertanyaan:

1. Apa perbedaan signature antara method yang di-override (canWithdraw(), printInfo()) dan method yang di-overload (deposit())?
   Jawab:
   - **Method yang di-override (canWithdraw(), printInfo()):** Wajib memiliki nama, jumlah, jenis parameter, dan tipe kembalian yang sama persis antara *superclass* dan *subclass*.
   - **Method yang di-overload (deposit()):** Berada dalam satu kelas yang sama dengan nama yang identik, tetapi jumlah atau tipe parameter (*signature*) harus berbeda.
2. Siapa yang menentukan versi method yang dijalankan pada overriding, dan siapa pada overloading?
   Jawab:
   - **Pada Overriding:** Ditentukan oleh **JVM saat *runtime*** berdasarkan tipe objek aktual yang dibuat di memori (*dynamic binding*).
   - **Pada Overloading:** Ditentukan oleh **Kompiler saat *compile-time*** berdasarkan argumen yang dilewatkan saat pemanggilan metode (*static binding*).

C. Hasil & Pembahasan

Pengujian program perbankan pada paket id.ac.polinema membuktikan keberhasilan penerapan Polimorfisme Dinamis (*Overriding*) dan Polimorfisme Statis (*Overloading*) berbasis Pewarisan (Inheritance). Melalui metode canWithdraw() yang di-override, setiap tipe akun dapat menerapkan aturan penarikan secara mandiri: SavingsAccount menolak penarikan jika saldo tersisa di bawah Rp50.000, CheckingAccount mengizinkan penarikan melebihi saldo memanfaatkan fasilitas cerukan (*overdraft*), dan BusinessAccount menjaga saldo terendah pada angka Rp1.000.000. Pemanggilan printInfo() pada objek yang disimpan di dalam kelas Bank juga dieksekusi secara tepat sesuai tipe objek aslinya berkat mekanisme *Dynamic Binding* oleh JVM. Sementara itu, penerapan *Method Overloading* pada deposit(double, String) berhasil memproses

penyetoran dana bersama catatan transaksi secara efisien yang ditangani langsung oleh kompiler saat *compile-time* (*Static Binding*).

D. Kesimpulan

Praktikum ini menjelaskan bahwa Polimorfisme dan Pewarisan merupakan fondasi utama OOP untuk menciptakan kode yang fleksibel dan modular. Melalui *Method Overriding*, subclass dapat menyesuaikan logika bisnis tanpa merusak struktur kelas induk, di mana eksekusinya ditentukan secara dinamis pada saat *runtime*. Sebaliknya, *Method Overloading* memberikan kemudahan variasi pemanggilan fungsi ber-parameter beda yang diselesaikan pada saat *compile-time*. Kedua teknik ini terbukti meningkatkan efisiensi kode, mencegah redundansi, serta mempermudah pengembangan sistem di masa mendatang.

E. Lampiran

Praktikum jobsheet 7:
https://github.com/Elsaamarthalinda/Praktikum-PBO/commit/66eaa248c79dd9acdecae76e4b622622344605c4

Tugas mandiri jobsheet 7:
https://github.com/Elsaamarthalinda/Praktikum-PBO/commit/031e2c692413bc5a472b6093c0b341cddfbc2fc2